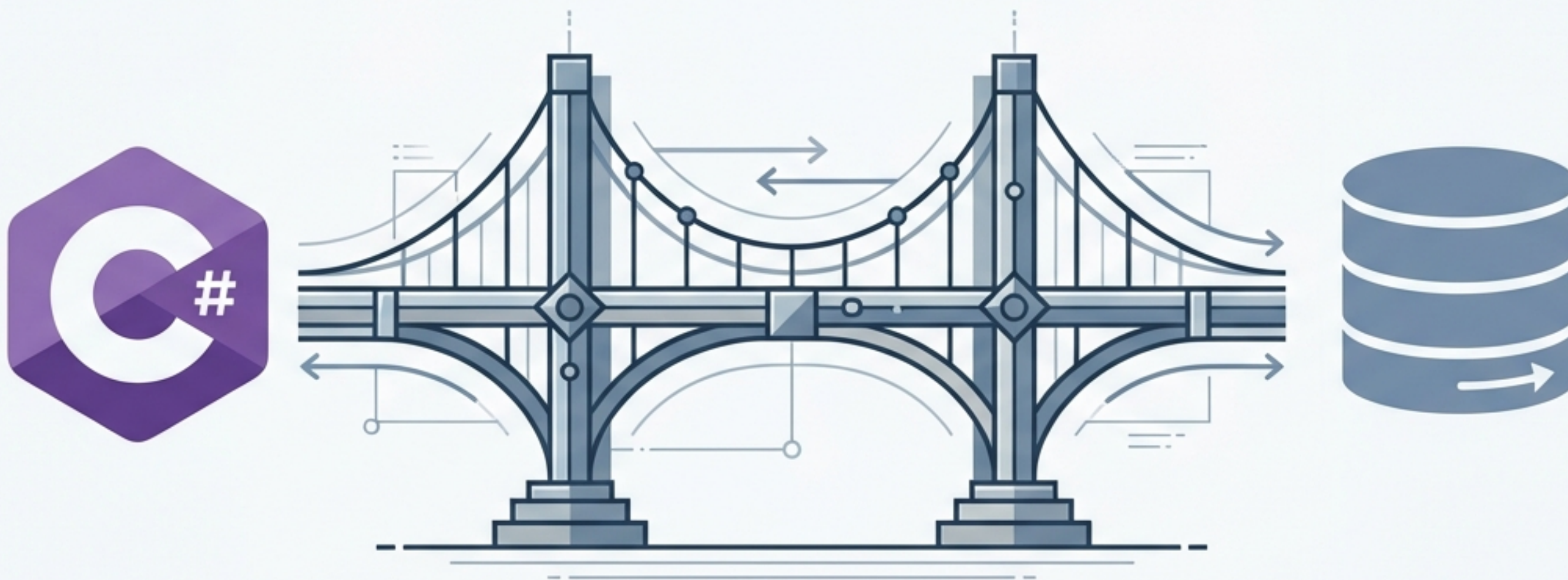


C# y SQLite: Integración Práctica



Bases de datos integradas sin configuración extra



Pequeño tamaño.

Un Sistema Gestor de Bases de Datos (SGBD) extremadamente ligero.



100% Autocontenido.

Se distribuye conjuntamente con el proyecto en un único archivo físico.

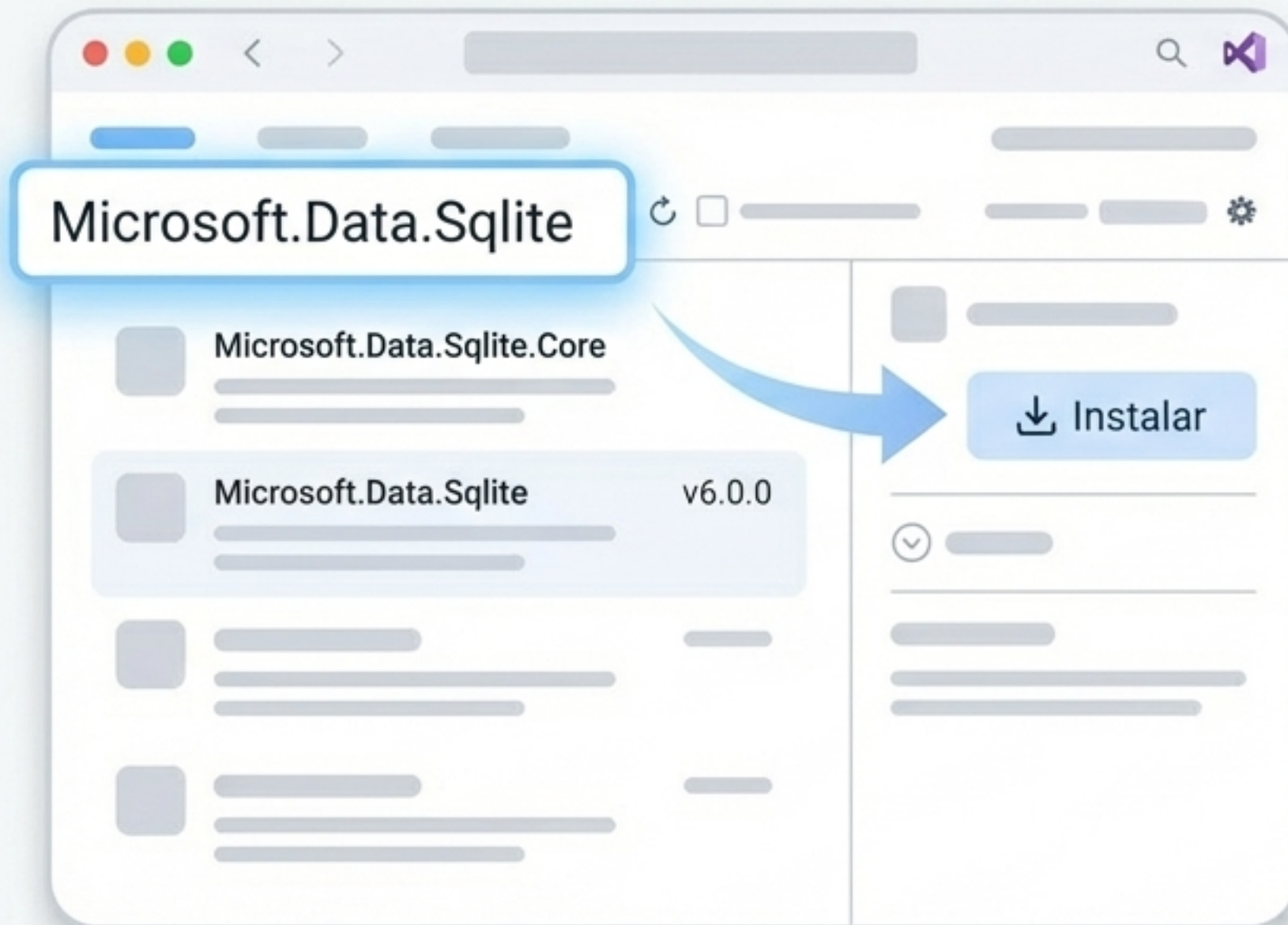


Cero configuración.

Ideal para aplicaciones móviles o de escritorio al no necesitar instalar software externo ni servidores.

Instalación del estándar actual

Olvida descargar archivos .dll manualmente. Hoy utilizamos el gestor de paquetes de .NET (NuGet) y la librería oficial Microsoft.Data.Sqlite.



```
dotnet new console -n AgendaModerna  
  
cd AgendaModerna  
  
dotnet add package Microsoft.Data.Sqlite
```


Los tres pilares de ADO.NET



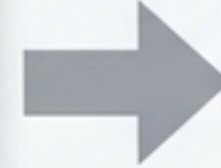
SqlConnection

Establece la conexión con el archivo de la base de datos.



SqlCommand

Representa la orden SQL que vamos a ejecutar.



SqlDataReader

Permite leer los resultados de una consulta SELECT fila por fila.

Gestión de Memoria (using)

Es vital instanciar estas clases dentro de bloques using. Esto garantiza que los recursos se liberen automáticamente de la memoria al terminar.

Creación de la base de datos y sus tablas

A diferencia de otros SGBD, las tablas se crean a mano con instrucciones SQL la primera vez que se ejecuta el programa.

Imprescindible abrir la conexión antes de operar.

Crea el archivo físico si no existe.

```
using (var conexion = new SqlConnection("Data Source=inventario.db"))
{
    conexion.Open();
    string sql = "CREATE TABLE Productos (Nombre TEXT, Precio REAL)";
    var comando = new SqlCommand(sql, conexion);
    comando.ExecuteNonQuery();
}
```

Consulta guardada como string para ser ejecutada después.

Prevenir inyección SQL mediante parámetros

Peligro



```
string sql = "SELECT * FROM Tabla  
WHERE id = " + variable;
```

Concatenar variables directamente expone tu programa a inyección SQL y hace el código confuso.

Correcto



```
string sql = "SELECT * FROM Tabla  
WHERE id = @parametro";  
  
comando.Parameters.AddWithValue  
("@parametro", variable);
```

Define marcas temporales y reemplázalas de forma segura especificando su valor exacto.

Insertar y modificar registros

```
string sql = "INSERT INTO Productos (Nombre) VALUES (@nombre)";  
var comando = new SqlCommand(sql, conexion);  
comando.Parameters.AddWithValue("@nombre", "Teclado Mecánico");  
int filas = comando.ExecuteNonQuery();  
Console.WriteLine($"{filas} fila(s) afectada(s)");
```



ExecuteNonQuery()

- Se utiliza para operaciones INSERT, UPDATE o DELETE.
- Devuelve un número entero: la cantidad de filas que han sido afectadas por la operación.

1 fila(s) afectada(s)

Leer datos registro a registro

```
string sql = "SELECT Nombre, Precio FROM Productos";  
var comando = new SqlCommand(sql, conexion);  
using (var reader = comando.ExecuteReader())  
using (var reader = comando.ExecuteReader())  
{  
    while (reader.Read())  
    {  
        Console.WriteLine(reader.GetString(0));  
    }  
}
```



1

Usamos ExecuteReader() para obtener un objeto SqlDataReader.

2

El método Read() avanza de registro en registro. El bucle while se ejecuta mientras sigan quedando filas por leer.



No olvides usar el método Close() si no estás usando bloques using automáticos.

Mapear registros a objetos en C#

En lugar de manejar datos sueltos en **arrays**, gestionamos los registros como colecciones de objetos.



A esta clase le podemos pasar la conexión a la base de datos como atributo para que gestione sus propias operaciones de actualización.

Retos prácticos: Iniciación

Nivel 1 (El primer contacto)

Objetivo

Crear inventario.db y tabla Productos (Nombre, Precio).

Tarea

Insertar tres productos con datos fijos directamente en el código usando consultas parametrizadas.

Nivel 2 (Interacción con el usuario)

Objetivo

Entrada dinámica de datos.

Tarea

Pedir al usuario nombre y precio por consola. Insertar el producto de forma segura y hacer un SELECT para imprimir todo el inventario.

Búsqueda y borrry borrado (Nivel 3)

Contexto: Base de datos Biblioteca con tabla Libros (Titulo, Autor).

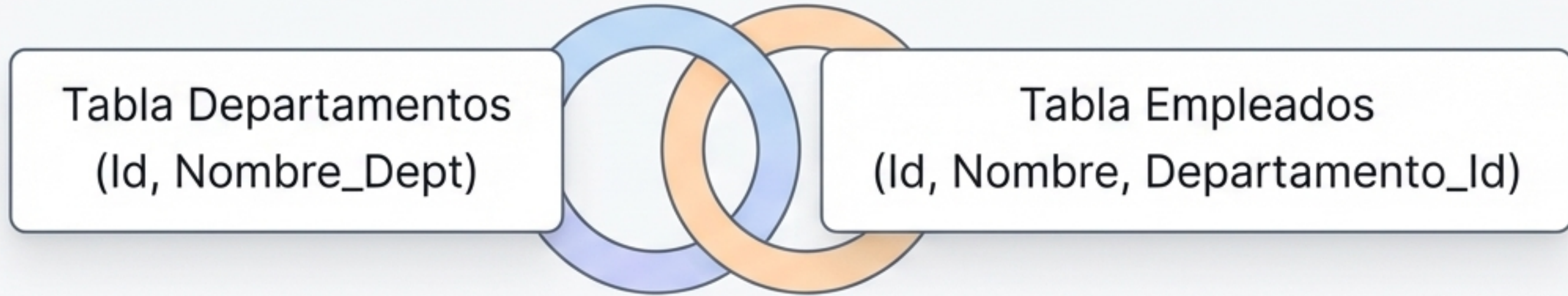
Misión 1

Preguntar por un autor. Mostrar solo sus libros utilizando la cláusula `WHERE` y parámetros de seguridad.

Misión 2

Preguntar por un título exacto. Eliminarlo usando `DELETE` y mostrar en pantalla cuántas filas fueron afectadas (usando el retorno de `ExecuteNonQuery`).

Relaciones modernas entre tablas (Nivel 4)



El Reto

Insertar datos cruzados y ejecutar una consulta SELECT compleja usando INNER JOIN.

Resultado esperado

El empleado [Nombre] trabaja en el departamento [Nombre_Dept]

El reto experto: CRUD Completo (Nivel 5)

Diseña un gestor de Contactos (Teléfono como Clave Primaria, Nombre, Email).

Menú cíclico

1. Añadir
2. Ver
3. Actualizar
4. Salir

Reglas Estrictas

⚠ **Bucle sin interrupciones:** Usar while con variable booleana (bool salir = false), absolutamente prohibido usar break.

⚠ **Menú Condicional:** Usar if-else if estricto, no usar switch.

⚠ **Gestor de Excepciones:** Usar try-catch. Si un teléfono existe, SQLite lanzará excepción. Capturarla para evitar el colapso del programa y mostrar un error amigable.

⚠ **Retorno Único:** Si usas funciones auxiliares, deben tener un único return al final.

Reto extra: Integración con Objetos C#

Contexto: Proyecto Ejercicio 10d (BBDD de Videojuegos)

1. Tabla

id autonumérico, título, precio (FLOAT/REAL). Cero nulos.

2. Requisito de Diseño

Crear la clase Videojuego para tratar cada registro de la base de datos como un objeto instanciado.

3. Menú a implementar

1. Añadir nuevo videojuego.
2. Listar catálogo completo.
3. Listar videojuegos más baratos que un precio dado.
4. Salir.

Resumen de herramientas clave

ExecuteNonQuery()	Uso: INSERT, UPDATE, DELETE. Devuelve: Número de filas afectadas.
ExecuteReader()	Uso: SELECT. Devuelve: SqlDataReader (para leer con Read()).
AddWithValue()	Uso: Seguridad. Función: Asigna valores reales a los parámetros marcados en la consulta SQL.
using { }	Uso: Memoria. Función: Libera y cierra conexiones o comandos automáticamente al terminar su bloque.